# Matrices and Systems of Linear Equations for Computer Scientists

Lecture Notes

A mathematical course with algorithmic and computational viewpoints

These notes are written for students of computer science. The main goal is not only to manipulate matrices correctly, but also to understand why matrices are central in algorithms, data representation, graphics, machine learning, optimization, networks, numerical computing, and scientific software.
The notes are purely mathematical in structure: definitions, propositions, examples, algorithms, and exercises. The computer science perspective appears as interpretation, motivation, and computational consequences.

## Contents


**1 Why matrices matter in computer science** **4**

1.1 A first example . . . . . 4

**2 Matrices, vectors, and notation** **5**

2.1 Shape discipline . . . . . 5

2.2 Special matrices . . . . . 5

**3 Vectors, linear combinations, span, and basis** **6**

3.1 Column space viewpoint . . . . . 6

3.2 Linear independence . . . . . 7

**4 Matrix arithmetic** **7**

4.1 Addition and scalar multiplication . . . . . 7

4.2 Matrix-vector multiplication . . . . . 7

4.3 Matrix-matrix multiplication . . . . . 8

4.4 Transpose . . . . . 8

**5 Matrices as linear transformations** **8**

5.1 The columns are images of basis vectors . . . . . 9

5.2 Composition . . . . . 9

**6 Linear systems: algebraic and geometric views** **10**

6.1 Three interpretations . . . . . 10

6.2 Possible numbers of solutions . . . 10

**7 Augmented matrices and elementary row operations 11**

7.1 Row echelon form . . . 11

7.2 A complete elimination example . . . 12

**8 Gaussian elimination as an algorithm 12**

8.1 Operation count . . . 13

8.2 Partial pivoting . . . 13

**9 Rank, pivots, and consistency 13**

9.1 Rank and systems . . . 14

9.2 Free variables . . . 14

**10 Homogeneous systems and null spaces 14**

10.1 Rank-nullity theorem . . . 15

10.2 General solution of a nonhomogeneous system . . . 15

**11 Inverse matrices 15**

11.1 The invertible matrix theorem . . . 15

**12 Determinants: what they measure 16**

12.1 Geometric meaning . . . 16

12.2 Determinants in computation . . . 16

**13 LU factorization 16**

13.1 Solving with LU . . . 17

13.2 Why factorization is useful . . . 17

**14 Triangular systems 17**

14.1 Back substitution formula . . . 17

**15 Numerical issues: floating point and conditioning 18**

15.1 Residual . . . 18

15.2 Conditioning . . . 18

**16 Orthogonality and projections 19**

16.1 Projection onto a line . . . 19

16.2 Orthogonal matrices . . . 19

**17 Least squares systems 19**

17.1 Normal equations . . . 20
17.2 Example: fitting a line . . . 20

**18 Sparse matrices** **21**
18.1 Storage formats . . . 21
18.2 Sparsity and graphs . . . 22

**19 Matrix decompositions beyond LU** **22**
19.1 QR factorization . . . 22
19.2 Cholesky factorization . . . 22
19.3 Singular value decomposition . . . 22

**20 Iterative methods** **23**
20.1 Jacobi method . . . 23
20.2 Gauss-Seidel method . . . 23
20.3 When iterative methods are useful . . . 23

**21 Matrices in data and machine learning** **23**
21.1 Feature dependence . . . 24
21.2 Regularization . . . 24

**22 Matrices in graphics and geometry** **24**
22.1 Change of basis . . . 24

**23 Matrix equations and block matrices** **25**
23.1 Schur complement . . . 25

**24 Complexity and practical implementation** **25**
24.1 Dense matrix operations . . . 25
24.2 Memory layout . . . 26
24.3 Do not reinvent everything . . . 26

**25 Worked mini-project: solving a small network balance problem** **26**

**26 Summary of key ideas** **26**

**27 Exercises** **27**

**Compact formula sheet** **29**

# 1 Why matrices matter in computer science

Matrices are one of the most important mathematical data structures in computer science. They appear whenever many numbers must be organized in a rectangular form or whenever a linear relationship must be computed efficiently.

A matrix can represent:

- a system of linear equations,
- a linear transformation of vectors,
- a dataset with samples as rows and features as columns,
- a graph by its adjacency matrix,
- an image as a grid of pixel intensities,
- a Markov chain by transition probabilities,
- a neural network layer, at least before applying a nonlinear activation,
- a sparse operator in numerical algorithms,
- constraints in optimization and linear programming.

The main object of this lecture is the linear system

$$A\boldsymbol{x} = \boldsymbol{b}.$$

Here $A$ is a matrix of coefficients, $\boldsymbol{x}$ is an unknown vector, and $\boldsymbol{b}$ is a known right-hand side. Much of linear algebra is the study of when this equation has a solution, how many solutions it has, how to compute them, and how stable the computation is.

**Computer science viewpoint**

In code, a linear system is not only a symbolic equation. It is a computational task. We must decide:

- how much memory is needed to store the matrix,
- how many arithmetic operations are needed,
- whether the algorithm is stable under rounding errors,
- whether the matrix is dense or sparse,
- whether the system has one solution, no solution, or infinitely many solutions,
- whether an approximate solution is enough.

## 1.1 A first example

Suppose three variables $x_1, x_2, x_3$ satisfy

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 1, \\ -x_1 + 3x_2 + 2x_3 &= 12, \\ 3x_1 - x_2 + x_3 &= 2. \end{aligned}$$

This system can be written compactly as

$$\begin{bmatrix} 2 & 1 & -1 \\ -1 & 3 & 2 \\ 3 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 12 \\ 2 \end{bmatrix}.$$

The compact notation is not only shorter. It exposes the structure of the problem. We can now apply general algorithms to the matrix rather than solving each system from scratch.

# 2 Matrices, vectors, and notation

**Definition**

An $m \times n$ **matrix** over $\mathbb{R}$ is a rectangular array of real numbers with $m$ rows and $n$ columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

The entry in row $i$ and column $j$ is denoted by $a_{ij}$. We write $A \in \mathbb{R}^{m \times n}$.

A column vector in $\mathbb{R}^n$ is an $n \times 1$ matrix:

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

A row vector is a $1 \times n$ matrix. In these notes, vectors are column vectors unless explicitly stated otherwise.

## 2.1 Shape discipline

In programming, matrices are arrays with shapes. In mathematics, the same rule is essential: an operation is meaningful only when the dimensions match.

| Operation | Condition | Resulting shape |
|---|---|---|
| $A + B$ | $A, B \in \mathbb{R}^{m \times n}$ | $m \times n$ |
| $cA$ | $c \in \mathbb{R}, A \in \mathbb{R}^{m \times n}$ | $m \times n$ |
| $A\boldsymbol{x}$ | $A \in \mathbb{R}^{m \times n},\ \boldsymbol{x} \in \mathbb{R}^n$ | $\mathbb{R}^m$ |
| $AB$ | $A \in \mathbb{R}^{m \times n},\ B \in \mathbb{R}^{n \times p}$ | $m \times p$ |
| $A^T$ | $A \in \mathbb{R}^{m \times n}$ | $n \times m$ |

**Warning**

Many algebraic errors are dimension errors. Before doing any calculation, check the shape of every object. This is the mathematical version of type checking.

## 2.2 Special matrices

Several matrices appear so often that they receive special names.

- The **zero matrix** has all entries equal to zero.
- The **identity matrix** $I_n$ is the $n \times n$ matrix with ones on the diagonal and zeros elsewhere:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

- A **diagonal matrix** has nonzero entries only possibly on the main diagonal.
- A **triangular matrix** has zeros either below or above the main diagonal.
- A **symmetric matrix** satisfies $A^T = A$.
- A **sparse matrix** has relatively few nonzero entries.

**Computer science viewpoint**

The distinction between dense and sparse matrices is crucial. A dense $10^6 \times 10^6$ matrix is impossible to store explicitly on ordinary hardware. But a sparse matrix of the same size with only a few nonzero entries per row may be practical.

# 3 Vectors, linear combinations, span, and basis

Matrices are built from vectors. The columns of a matrix are especially important:

$$A = \begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \end{bmatrix}.$$

Then matrix-vector multiplication can be read as a linear combination of columns:

$$A\boldsymbol{x} = x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n.$$

**Definition**

A **linear combination** of vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ is a vector of the form

$$c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_k\boldsymbol{v}_k,$$

where $c_1, \ldots, c_k$ are scalars.
The **span** of $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$, denoted $\operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$, is the set of all their linear combinations.

## 3.1 Column space viewpoint

The equation $A\boldsymbol{x} = \boldsymbol{b}$ asks whether $\boldsymbol{b}$ is a linear combination of the columns of $A$:

$$\boldsymbol{b} \in \operatorname{span}\{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n\}.$$

If yes, the system is consistent. If not, it has no exact solution.

**Theorem / Principle**

The linear system $A\boldsymbol{x} = \boldsymbol{b}$ has at least one solution if and only if $\boldsymbol{b}$ belongs to the column space of $A$.

**Example**

Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}, \qquad \boldsymbol{b} = \begin{bmatrix} 5 \\ 15 \end{bmatrix}.$$

The columns of $A$ are $\boldsymbol{a}_1 = (1, 3)^T$ and $\boldsymbol{a}_2 = (2, 6)^T = 2\boldsymbol{a}_1$. The column space is a line. Since $\boldsymbol{b} = 5\boldsymbol{a}_1$, the system is consistent.
If instead $\boldsymbol{b} = (5, 14)^T$, the vector is not on that line, so the system is inconsistent.

### 3.2 Linear independence

**Definition**

Vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ are **linearly independent** if

$$c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0}$$

implies

$$c_1 = \cdots = c_k = 0.$$

If a nontrivial combination gives zero, the vectors are **linearly dependent**.

Linear independence means no vector in the list is redundant. In data terms, one feature cannot be exactly reconstructed from the others. In geometry, independent vectors give genuinely new directions.

## 4 Matrix arithmetic

### 4.1 Addition and scalar multiplication

If $A, B \in \mathbb{R}^{m\times n}$, then

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

If $c \in \mathbb{R}$, then

$$(cA)_{ij} = ca_{ij}.$$

These operations are entrywise and have the same algebraic properties as vector addition.

### 4.2 Matrix-vector multiplication

Let

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}, \qquad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

Then

$$A\boldsymbol{x} = \begin{bmatrix} \sum_{j=1}^n a_{1j}x_j \\ \vdots \\ \sum_{j=1}^n a_{mj}x_j \end{bmatrix}.$$

Each row of $A$ defines one dot product.

**Computer science viewpoint**

A dense matrix-vector product with $A \in \mathbb{R}^{m\times n}$ takes about $mn$ multiplications and $m(n-1)$ additions. A sparse matrix-vector product is proportional to the number of nonzero entries, often written $\mathrm{nnz}(A)$.

### 4.3 Matrix-matrix multiplication

If $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$, then $AB \in \mathbb{R}^{m\times p}$ and

$$(AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

The entry $(AB)_{ij}$ is the dot product of row $i$ of $A$ and column $j$ of $B$.

**Warning**

Matrix multiplication is generally not commutative. Usually $AB \neq BA$, and sometimes one product is defined while the other is not.

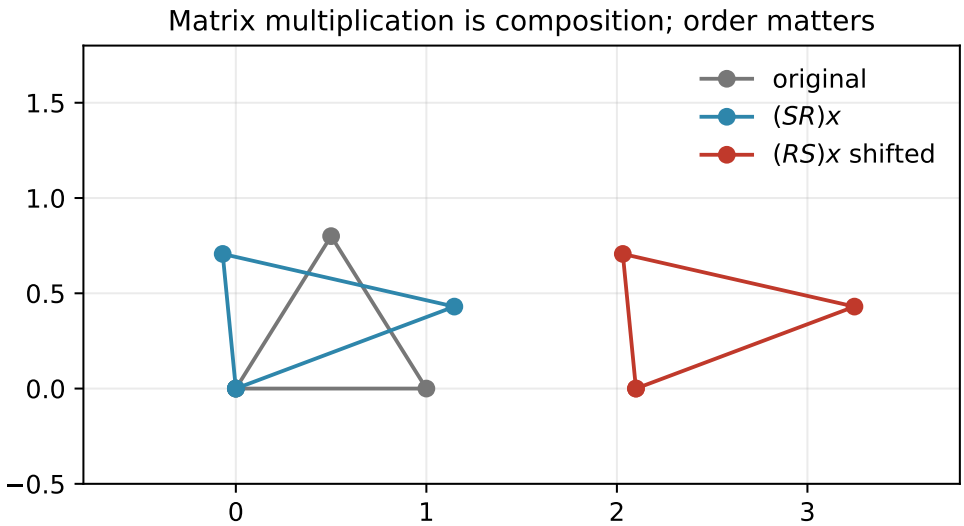


Figure 1: Matrix multiplication represents composition of transformations. The order of composition matters.

### 4.4 Transpose

The transpose of $A \in \mathbb{R}^{m\times n}$ is the matrix $A^T \in \mathbb{R}^{n\times m}$ defined by

$$(A^T)_{ij} = A_{ji}.$$

Important identities:

$$(A+B)^T = A^T + B^T, \qquad (cA)^T = cA^T, \qquad (AB)^T = B^T A^T.$$

The last formula reverses the order. This is a common source of mistakes.

## 5 Matrices as linear transformations

A function $T : \mathbb{R}^n \to \mathbb{R}^m$ is linear if

$$T(\boldsymbol{x}+\boldsymbol{y}) = T(\boldsymbol{x}) + T(\boldsymbol{y}), \qquad T(c\boldsymbol{x}) = cT(\boldsymbol{x}).$$

Every matrix $A \in \mathbb{R}^{m\times n}$ defines a linear map

$$T_A(\boldsymbol{x}) = A\boldsymbol{x}.$$

Conversely, every linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$ can be represented by a matrix once bases are chosen.

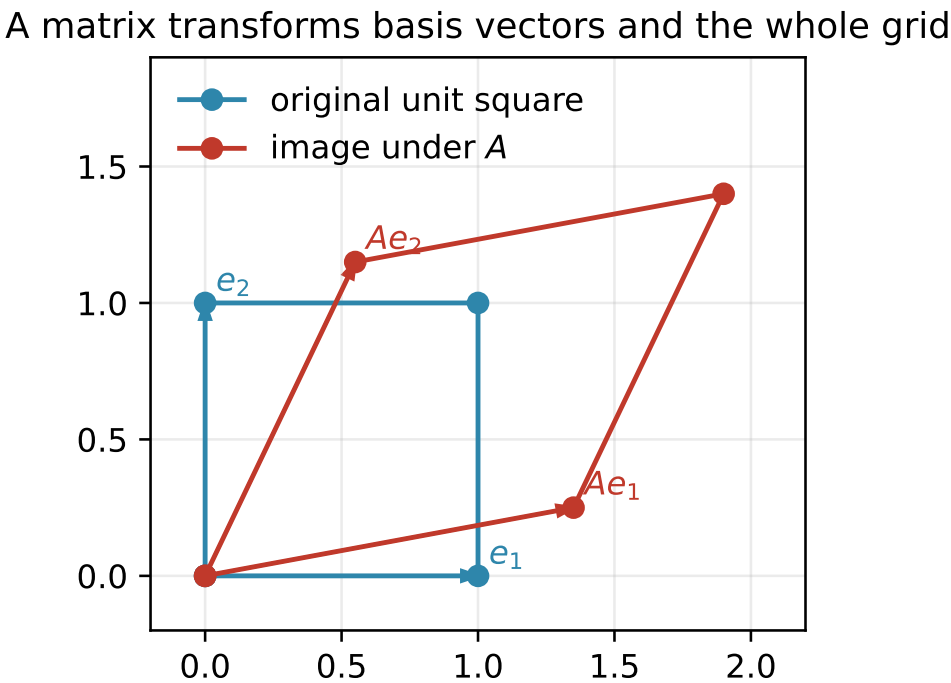


Figure 2: A matrix is determined by where it sends the basis vectors. The unit square becomes a parallelogram.

### 5.1 The columns are images of basis vectors

Let $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n$ be the standard basis vectors. If

$$A = \begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \end{bmatrix},$$

then

$$A\boldsymbol{e}_j = \boldsymbol{a}_j.$$

Thus the columns of $A$ tell us where the basis vectors go.

**Computer science viewpoint**

In computer graphics, transformations such as scaling, rotation, shearing, projection, and change of coordinates are implemented by matrices. In homogeneous coordinates, translations can also be represented by matrices.

### 5.2 Composition

If $A : \mathbb{R}^n \to \mathbb{R}^m$ and $B : \mathbb{R}^p \to \mathbb{R}^n$, then

$$A(B\boldsymbol{x}) = (AB)\boldsymbol{x}.$$

So matrix multiplication is composition of linear transformations.

This viewpoint is often more meaningful than the entry formula. It explains why multiplication is associative:

$$A(BC) = (AB)C,$$

because function composition is associative.

# 6 Linear systems: algebraic and geometric views

The general linear system with $m$ equations and $n$ unknowns is

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m. \end{aligned}$$

In matrix form:

$$A\boldsymbol{x} = \boldsymbol{b}.$$

## 6.1 Three interpretations

1. **Row view:** Each equation describes a hyperplane. The solution set is the intersection of these hyperplanes.
2. **Column view:** The equation asks whether $\boldsymbol{b}$ is a linear combination of the columns of $A$.
3. **Transformation view:** We search for an input $\boldsymbol{x}$ such that the linear map $\boldsymbol{x} \mapsto A\boldsymbol{x}$ produces $\boldsymbol{b}$.

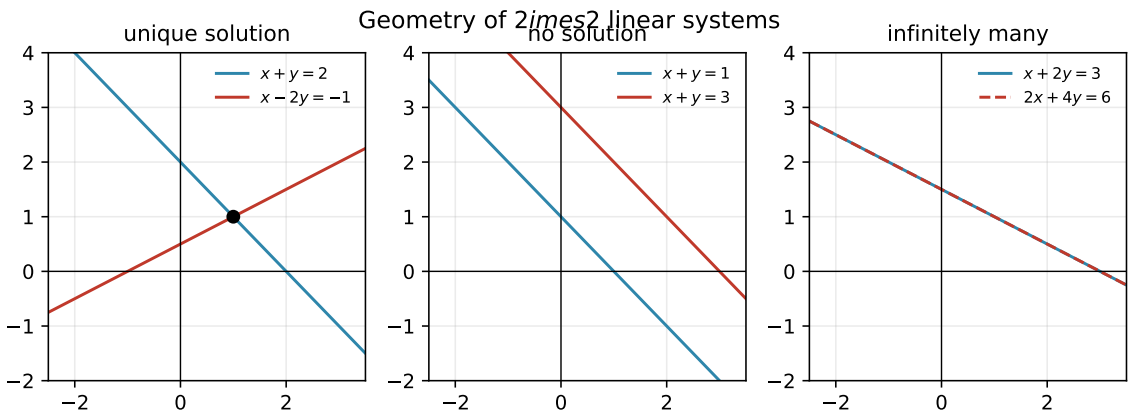


Figure 3: In two variables, linear equations are lines. Their intersection pattern determines the number of solutions.

## 6.2 Possible numbers of solutions

A linear system can have:

- no solution,
- exactly one solution,

- infinitely many solutions.

It cannot have exactly two solutions. The reason is geometric: solution sets of linear systems are affine subspaces, not arbitrary sets.

**Theorem / Principle**

If a linear system has two distinct solutions, then it has infinitely many solutions.

**Proof.** Suppose $\boldsymbol{x}$ and $\boldsymbol{y}$ are distinct solutions of $A\boldsymbol{x} = \boldsymbol{b}$. Then for any $t \in \mathbb{R}$, define

$$\boldsymbol{z}_t = t\boldsymbol{x} + (1-t)\boldsymbol{y}.$$

Then

$$A\boldsymbol{z}_t = tA\boldsymbol{x} + (1-t)A\boldsymbol{y} = t\boldsymbol{b} + (1-t)\boldsymbol{b} = \boldsymbol{b}.$$

So every point on the line through $\boldsymbol{x}$ and $\boldsymbol{y}$ is also a solution. □

# 7 Augmented matrices and elementary row operations

To solve $A\boldsymbol{x} = \boldsymbol{b}$, we write the augmented matrix

$$[A \mid \boldsymbol{b}].$$

For example,

$$\begin{aligned} 2x + y &= 5, \\ x - y &= 1 \end{aligned} \qquad \Longleftrightarrow \qquad \left[\begin{array}{cc|c} 2 & 1 & 5 \\ 1 & -1 & 1 \end{array}\right].$$

**Definition**

The elementary row operations are:

1. swap two rows,
2. multiply a row by a nonzero scalar,
3. add a scalar multiple of one row to another row.

These operations preserve the solution set of a linear system. Therefore they may be used to simplify the system without changing its answers.

## 7.1 Row echelon form

**Definition**

A matrix is in **row echelon form** if:

1. all zero rows, if any, are at the bottom,
2. each nonzero row has a leading nonzero entry called a pivot,
3. pivots move strictly to the right as we go down the rows,
4. entries below each pivot are zero.

It is in **reduced row echelon form** if, in addition, each pivot is equal to 1 and is the only nonzero entry in its column.

### 7.2 A complete elimination example

**Example**

Solve

$$\begin{aligned} x + 2y - z &= 3, \\ 2x + 5y + z &= 12, \\ -x - y + 2z &= -1. \end{aligned}$$

The augmented matrix is

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 2 & 5 & 1 & 12 \\ -1 & -1 & 2 & -1 \end{array}\right].$$

Eliminate below the first pivot:

$$R_2 \leftarrow R_2 - 2R_1, \qquad R_3 \leftarrow R_3 + R_1.$$

Then

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & 1 & 3 & 6 \\ 0 & 1 & 1 & 2 \end{array}\right].$$

Eliminate below the second pivot:

$$R_3 \leftarrow R_3 - R_2.$$

So

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & 1 & 3 & 6 \\ 0 & 0 & -2 & -4 \end{array}\right].$$

Back substitution gives

$$-2z = -4 \Rightarrow z = 2,$$

$$y + 3z = 6 \Rightarrow y = 0,$$

$$x + 2y - z = 3 \Rightarrow x = 5.$$

Therefore

$$\boldsymbol{x} = \begin{bmatrix} 5 \\ 0 \\ 2 \end{bmatrix}.$$

## 8 Gaussian elimination as an algorithm

Gaussian elimination converts a linear system into an equivalent triangular system. It has two stages:

1. **Forward elimination:** create zeros below the pivots.
2. **Back substitution:** solve from the last equation upward.

**Algorithmic form**

```
Input: A in R^{n x n}, b in R^n
Output: x satisfying Ax = b, if the algorithm succeeds

for k = 1 to n-1:
    choose pivot in column k
    for i = k+1 to n:
        multiplier = A[i,k] / A[k,k]
        for j = k to n:
            A[i,j] = A[i,j] - multiplier * A[k,j]
        b[i] = b[i] - multiplier * b[k]

solve the resulting upper triangular system by back substitution
```

### 8.1 Operation count

For a dense $n \times n$ system, Gaussian elimination requires approximately

$$\frac{2}{3}n^3$$

floating point operations for the elimination step. Back substitution costs about $n^2$ operations. Thus the leading order cost is $O(n^3)$.

**Computer science viewpoint**

The asymptotic complexity $O(n^3)$ matters, but constants and memory access also matter. Modern linear algebra libraries are optimized around cache-friendly block operations and highly tuned matrix multiplication kernels.

### 8.2 Partial pivoting

If a pivot is zero, elimination cannot proceed without swapping rows. Even if the pivot is very small, division by it can amplify rounding errors. Therefore practical implementations use pivoting.

**Definition**

**Partial pivoting** means that, before eliminating column $k$, we swap the current row with a lower row whose entry in column $k$ has maximal absolute value.

Pivoting is both an algebraic and numerical tool. It avoids division by zero and improves stability.

## 9 Rank, pivots, and consistency

**Definition**

The **rank** of a matrix $A$, denoted $\operatorname{rank}(A)$, is the number of pivots in its row echelon form. Equivalently, it is the dimension of the column space of $A$.

Rank measures the amount of independent information in a matrix.

### 9.1 Rank and systems

For a system $A\boldsymbol{x} = \boldsymbol{b}$, compare the rank of $A$ with the rank of the augmented matrix $[A \mid \boldsymbol{b}]$.

**Theorem / Principle**

The system $A\boldsymbol{x} = \boldsymbol{b}$ is consistent if and only if

$$\operatorname{rank}(A) = \operatorname{rank}([A \mid \boldsymbol{b}]).$$

If the system is consistent and $A \in \mathbb{R}^{m\times n}$, then:

- if $\operatorname{rank}(A) = n$, the solution is unique,
- if $\operatorname{rank}(A) < n$, there are infinitely many solutions.

**Example**

Consider

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 4 \\ 2 & 4 & 2 & 8 \\ 1 & 1 & 0 & 1 \end{array}\right].$$

The second equation is twice the first, so it carries no new information. Row reduction gives two pivots, not three. Since there are three unknowns and the system is consistent, there is one free variable and infinitely many solutions.

### 9.2 Free variables

Variables corresponding to pivot columns are called pivot variables. Variables corresponding to non-pivot columns are free variables.

If $n$ unknowns and $r$ pivots occur, then there are $n - r$ free variables. This number is the dimension of the solution space of the homogeneous system.

## 10 Homogeneous systems and null spaces

A homogeneous system has the form

$$A\boldsymbol{x} = \boldsymbol{0}.$$

It always has at least one solution, namely $\boldsymbol{x} = \boldsymbol{0}$. This is called the trivial solution.

**Definition**

The **null space** of $A$ is

$$\operatorname{Null}(A) = \{\boldsymbol{x} \in \mathbb{R}^n : A\boldsymbol{x} = \boldsymbol{0}\}.$$

The null space contains all input directions destroyed by the transformation $\boldsymbol{x} \mapsto A\boldsymbol{x}$.

**Computer science viewpoint**

In computer science, null spaces appear in constraint solving, data compression, graphics, coding theory, and optimization. A nonzero vector in the null space represents a change in variables that leaves the output unchanged.

### 10.1 Rank-nullity theorem

**Theorem / Principle**

If $A \in \mathbb{R}^{m\times n}$, then

$$\operatorname{rank}(A) + \dim \operatorname{Null}(A) = n.$$

This is the rank-nullity theorem.

The theorem says that every input direction either contributes to the output space or belongs to the null space, with the correct accounting of dimensions.

### 10.2 General solution of a nonhomogeneous system

If $A\boldsymbol{x} = \boldsymbol{b}$ is consistent and $\boldsymbol{x}_p$ is one particular solution, then all solutions have the form

$$\boldsymbol{x} = \boldsymbol{x}_p + \boldsymbol{x}_h,$$

where $\boldsymbol{x}_h \in \operatorname{Null}(A)$.

So the solution set is a shifted copy of the null space.

## 11 Inverse matrices

**Definition**

A square matrix $A \in \mathbb{R}^{n\times n}$ is **invertible** if there exists a matrix $A^{-1} \in \mathbb{R}^{n\times n}$ such that

$$A^{-1}A = AA^{-1} = I_n.$$

If $A$ is invertible, then the system $A\boldsymbol{x} = \boldsymbol{b}$ has the unique solution

$$\boldsymbol{x} = A^{-1}\boldsymbol{b}.$$

**Warning**

Although the formula $\boldsymbol{x} = A^{-1}\boldsymbol{b}$ is mathematically correct, one usually does not compute $A^{-1}$ explicitly to solve a system. Numerical software solves the system using factorizations such as LU, QR, or Cholesky, depending on the matrix.

### 11.1 The invertible matrix theorem

**Theorem / Principle**

For $A \in \mathbb{R}^{n\times n}$, the following statements are equivalent:

1. $A$ is invertible.
2. $A\boldsymbol{x} = \boldsymbol{b}$ has a unique solution for every $\boldsymbol{b} \in \mathbb{R}^n$.
3. $A\boldsymbol{x} = \boldsymbol{0}$ has only the trivial solution.
4. The columns of $A$ are linearly independent.
5. The columns of $A$ span $\mathbb{R}^n$.
6. $\operatorname{rank}(A) = n$.

7. The row echelon form of $A$ has a pivot in every column.

This theorem is central because it connects algebra, geometry, and computation.

# 12 Determinants: what they measure

The determinant is a scalar associated with a square matrix. For a $2 \times 2$ matrix,

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \det(A) = ad - bc.$$

## 12.1 Geometric meaning

In $\mathbb{R}^2$, $|\det(A)|$ is the area scaling factor of the linear transformation $\boldsymbol{x} \mapsto A\boldsymbol{x}$. In $\mathbb{R}^3$, it is the volume scaling factor. The sign gives orientation.

**Theorem / Principle**

A square matrix $A$ is invertible if and only if

$$\det(A) \neq 0.$$

## 12.2 Determinants in computation

For theoretical work, determinants are useful. For numerical solution of linear systems, they are usually not the best tool. Checking invertibility by computing a determinant can be less stable and less informative than using rank-revealing factorization.

**Computer science viewpoint**

In computational linear algebra, determinants are often replaced by more stable concepts: pivot sizes, singular values, condition numbers, and factorization diagnostics.

# 13 LU factorization

Gaussian elimination can be reorganized as a matrix factorization.

**Definition**

An **LU factorization** of a square matrix $A$ is a decomposition

$$A = LU,$$

where $L$ is lower triangular and $U$ is upper triangular.

If pivoting is needed, we usually have

$$PA = LU,$$

where $P$ is a permutation matrix representing row swaps.

### 13.1 Solving with LU

If $A = LU$, then $A\boldsymbol{x} = \boldsymbol{b}$ becomes

$$LU\boldsymbol{x} = \boldsymbol{b}.$$

Set $U\boldsymbol{x} = \boldsymbol{y}$. Then solve two triangular systems:

$$L\boldsymbol{y} = \boldsymbol{b}, \qquad U\boldsymbol{x} = \boldsymbol{y}.$$

The first is solved by forward substitution; the second by back substitution.

**Algorithmic form**

```
Input: LU factorization A = LU and right-hand side b
Output: solution x of Ax = b

solve L y = b by forward substitution
solve U x = y by back substitution
return x
```

### 13.2 Why factorization is useful

If we need to solve many systems with the same matrix but different right-hand sides,

$$A\boldsymbol{x}^{(1)} = \boldsymbol{b}^{(1)}, \quad A\boldsymbol{x}^{(2)} = \boldsymbol{b}^{(2)}, \quad \ldots,$$

then we factorize $A$ once. Each new right-hand side requires only triangular solves.

**Computer science viewpoint**

This is common in simulation, optimization, statistics, and data processing. The expensive part is the factorization. Once it is available, repeated solves are much cheaper.

## 14 Triangular systems

A triangular system is much easier to solve than a general system.

For example,

$$\begin{bmatrix} 2 & -1 & 3 \\ 0 & 4 & 1 \\ 0 & 0 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 7 \\ 6 \\ 10 \end{bmatrix}.$$

The last equation gives $5x_3 = 10$, so $x_3 = 2$. Then the second equation gives $4x_2 + x_3 = 6$, so $x_2 = 1$. Finally the first equation gives $2x_1 - x_2 + 3x_3 = 7$, so $x_1 = 1$.

### 14.1 Back substitution formula

For an upper triangular system $U\boldsymbol{x} = \boldsymbol{b}$, the equations are

$$\sum_{j=i}^{n} u_{ij} x_j = b_i, \qquad i = 1, \ldots, n.$$

Solving backward gives

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}.$$

This requires $u_{ii} \neq 0$.

# 15 Numerical issues: floating point and conditioning

Mathematical algebra assumes exact arithmetic. Computers use floating point arithmetic, which introduces rounding errors. Therefore a computed solution $\hat{\boldsymbol{x}}$ may not be exactly equal to the true solution $\boldsymbol{x}$.

## 15.1 Residual

The residual of an approximate solution $\hat{\boldsymbol{x}}$ is

$$\boldsymbol{r} = \boldsymbol{b} - A\hat{\boldsymbol{x}}.$$

A small residual means that $\hat{\boldsymbol{x}}$ nearly satisfies the equation. However, a small residual does not always imply a small error in $\hat{\boldsymbol{x}}$.

## 15.2 Conditioning

A problem is well-conditioned if small changes in the input lead to small changes in the output. It is ill-conditioned if small input perturbations can cause large output changes.

For an invertible matrix $A$, the condition number with respect to a matrix norm is

$$\operatorname{cond}(A) = \|A\| \left\|A^{-1}\right\|.$$

Large condition numbers indicate potential sensitivity.

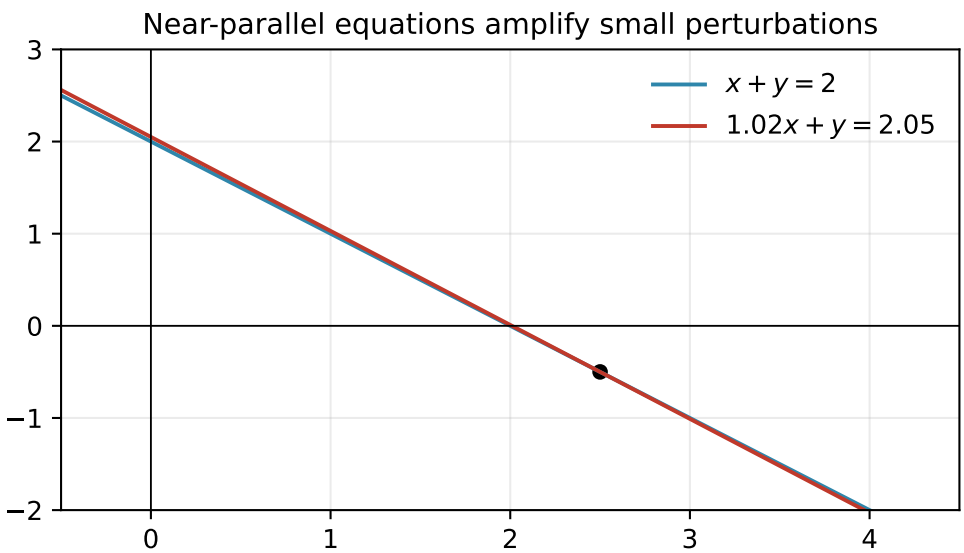


Figure 4: When equations are almost dependent, small changes can move the intersection point significantly.

**Warning**

Ill-conditioning is not the same as using a bad algorithm. An ill-conditioned problem is inherently sensitive. A bad algorithm can make the result worse, but even a good algorithm cannot remove the underlying sensitivity.

# 16 Orthogonality and projections

The dot product of $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ is

$$\boldsymbol{x}^T \boldsymbol{y} = \sum_{i=1}^{n} x_i y_i.$$

The Euclidean norm is

$$\|\boldsymbol{x}\|_2 = \sqrt{\boldsymbol{x}^T \boldsymbol{x}}.$$

Two vectors are orthogonal if

$$\boldsymbol{x}^T \boldsymbol{y} = 0.$$

## 16.1 Projection onto a line

If $\boldsymbol{a} \neq \boldsymbol{0}$, the projection of $\boldsymbol{b}$ onto the line spanned by $\boldsymbol{a}$ is

$$\operatorname{proj}_{\boldsymbol{a}}(\boldsymbol{b}) = \frac{\boldsymbol{a}^T \boldsymbol{b}}{\boldsymbol{a}^T \boldsymbol{a}} \boldsymbol{a}.$$

The error vector $\boldsymbol{b} - \operatorname{proj}_{\boldsymbol{a}}(\boldsymbol{b})$ is orthogonal to $\boldsymbol{a}$.

## 16.2 Orthogonal matrices

**Definition**

A square matrix $Q$ is **orthogonal** if

$$Q^T Q = I.$$

Equivalently, the columns of $Q$ form an orthonormal basis.

Orthogonal matrices preserve lengths and angles:

$$\|Q\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2 .$$

This makes them numerically valuable.

# 17 Least squares systems

In many applications, a system has more equations than unknowns:

$$A\boldsymbol{x} \approx \boldsymbol{b}, \qquad A \in \mathbb{R}^{m \times n}, \quad m > n.$$

Usually there is no exact solution. We then seek the vector $\hat{\boldsymbol{x}}$ that minimizes the squared error:

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x} \in \mathbb{R}^n} \|A\boldsymbol{x} - \boldsymbol{b}\|_2^2 .$$

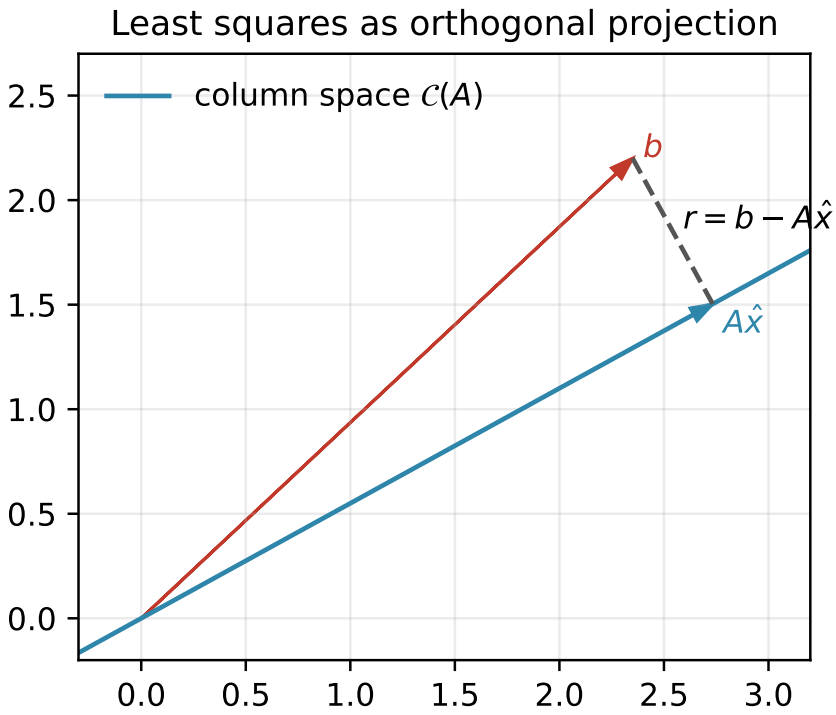


Figure 5: Least squares chooses the point in the column space of $A$ closest to $\boldsymbol{b}$.

### 17.1 Normal equations

At the minimizer, the residual

$$\boldsymbol{r} = \boldsymbol{b} - A\hat{\boldsymbol{x}}$$

is orthogonal to every column of $A$. Hence

$$A^T(\boldsymbol{b} - A\hat{\boldsymbol{x}}) = \boldsymbol{0}.$$

Therefore

$$A^T A\hat{\boldsymbol{x}} = A^T\boldsymbol{b}.$$

These are the normal equations.

**Warning**

The normal equations are mathematically elegant, but they can square the condition number. In numerical computation, QR factorization or singular value decomposition is often preferred for least squares problems.

### 17.2 Example: fitting a line

Suppose we have data points $(t_i, y_i)$ and want to fit a line

$$y \approx c_0 + c_1 t.$$

For each data point,

$$c_0 + c_1 t_i \approx y_i.$$

This gives the matrix equation

$$\begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Thus linear regression is a least squares problem.

# 18 Sparse matrices

A matrix is sparse if most of its entries are zero. Sparsity is not only a property of the matrix; it is an opportunity for efficient algorithms.

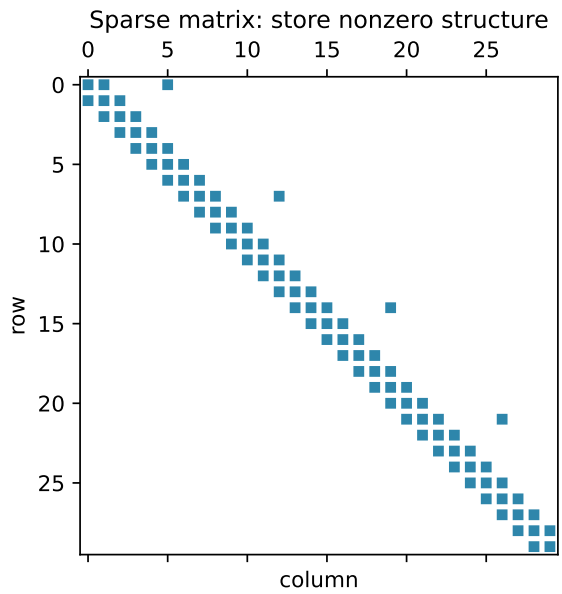


Figure 6: A sparse matrix is best understood by its nonzero pattern.

## 18.1 Storage formats

Instead of storing all entries, sparse matrices store only nonzero values and their positions. Common formats include:

- coordinate format (COO),
- compressed sparse row (CSR),
- compressed sparse column (CSC),
- diagonal formats for banded matrices.

**Computer science viewpoint**

If a matrix has $n$ rows and only about $k$ nonzero entries per row, sparse matrix-vector multiplication costs about $O(kn)$, not $O(n^2)$. This is essential in graph algorithms, finite difference methods, search ranking, and machine learning.

### 18.2 Sparsity and graphs

An undirected graph with $n$ vertices can be represented by an adjacency matrix $A$, where

$$a_{ij} = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are connected,} \\ 0, & \text{otherwise.} \end{cases}$$

If the graph has few edges compared with $n^2$, its adjacency matrix is sparse.

Matrix powers also encode paths. For an adjacency matrix $A$, the entry $(A^k)_{ij}$ counts the number of walks of length $k$ from vertex $i$ to vertex $j$, for unweighted graphs.

## 19 Matrix decompositions beyond LU

Different computational tasks require different decompositions.

### 19.1 QR factorization

A QR factorization writes

$$A = QR,$$

where $Q$ has orthonormal columns and $R$ is upper triangular. QR is especially important for least squares problems because orthogonal transformations are numerically stable.

### 19.2 Cholesky factorization

If $A$ is symmetric positive definite, then

$$A = LL^T,$$

where $L$ is lower triangular. This is the Cholesky factorization. It is faster and more memory-efficient than general LU for suitable matrices.

### 19.3 Singular value decomposition

The singular value decomposition writes

$$A = U\Sigma V^T,$$

where $U$ and $V$ are orthogonal and $\Sigma$ is diagonal with nonnegative entries. The diagonal entries are singular values.

SVD is central in low-rank approximation, principal component analysis, recommendation systems, numerical rank detection, and dimensionality reduction.

**Computer science viewpoint**

The choice of factorization is an algorithmic design decision. The same mathematical problem can lead to different algorithms depending on matrix structure: dense, sparse, symmetric, positive definite, rectangular, ill-conditioned, or low-rank.

# 20 Iterative methods

For very large systems, direct factorization may be too expensive. Iterative methods build a sequence of approximations

$$\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots$$

that hopefully converges to the solution.

## 20.1 Jacobi method

Split

$$A = D + L + U,$$

where $D$ is diagonal, $L$ is strictly lower triangular, and $U$ is strictly upper triangular. The Jacobi iteration is

$$\boldsymbol{x}^{(k+1)} = D^{-1}\left(\boldsymbol{b} - (L+U)\boldsymbol{x}^{(k)}\right).$$

This method is simple and parallelizable, but it does not always converge.

## 20.2 Gauss-Seidel method

The Gauss-Seidel iteration is

$$(D+L)\boldsymbol{x}^{(k+1)} = \boldsymbol{b} - U\boldsymbol{x}^{(k)}.$$

It uses newly updated values immediately and often converges faster than Jacobi.

## 20.3 When iterative methods are useful

Iterative methods are useful when:

- the system is very large,
- the matrix is sparse,
- matrix-vector multiplication is cheap,
- an approximate solution is sufficient,
- storing a full factorization is impractical.

# 21 Matrices in data and machine learning

In data science, a dataset is often represented as a matrix

$$X \in \mathbb{R}^{m \times n},$$

where rows are observations and columns are features.

A linear model has the form

$$\hat{\boldsymbol{y}} = X\boldsymbol{w}.$$

Training by least squares means finding $\boldsymbol{w}$ minimizing

$$\|X\boldsymbol{w} - \boldsymbol{y}\|_2^2.$$

### 21.1 Feature dependence

If columns of $X$ are linearly dependent, then some features are redundant. This can cause non-uniqueness or numerical instability in the fitted coefficients.

### 21.2 Regularization

To stabilize a least squares problem, one often solves

$$\min_{\boldsymbol{w}} \left( \|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2 \right), \qquad \lambda > 0.$$

The normal equations become

$$(X^T X + \lambda I)\boldsymbol{w} = X^T \boldsymbol{y}.$$

This is ridge regression.

**Computer science viewpoint**

Regularization changes the linear system in a way that improves stability and controls overfitting. The matrix $X^T X + \lambda I$ is often better conditioned than $X^T X$.

## 22 Matrices in graphics and geometry

Two-dimensional and three-dimensional graphics rely heavily on matrix transformations.

Common $2 \times 2$ matrices include:

$$\text{Rotation: } R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix},$$

$$\text{Scaling: } S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix},$$

$$\text{Shear: } H = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}.$$

A point $\boldsymbol{x} \in \mathbb{R}^2$ is transformed by $\boldsymbol{x}' = A\boldsymbol{x}$. In homogeneous coordinates, a point $(x, y)$ is represented as $(x, y, 1)$, and affine transformations become $3 \times 3$ matrices.

### 22.1 Change of basis

If a vector has coordinates $[\boldsymbol{x}]_B$ in a basis $B = (\boldsymbol{b}_1, \ldots, \boldsymbol{b}_n)$, and

$$P = \begin{bmatrix} \boldsymbol{b}_1 & \cdots & \boldsymbol{b}_n \end{bmatrix},$$

then its standard coordinates are

$$\boldsymbol{x} = P[\boldsymbol{x}]_B.$$

Changing coordinates is solving a linear system.

# 23 Matrix equations and block matrices

Matrices may be partitioned into blocks. This is useful when a problem naturally has subcomponents.

For example,

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

Block multiplication follows the same rules as ordinary multiplication, provided dimensions match:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}.$$

**Computer science viewpoint**

Block matrices are important in high-performance computing. Algorithms are often reorganized into block operations because they use memory caches more efficiently and call optimized matrix kernels.

## 23.1 Schur complement

If $A$ is invertible, the block system

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{bmatrix} = \begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{g} \end{bmatrix}$$

can be reduced. From the first block row,

$$\boldsymbol{x} = A^{-1}(\boldsymbol{f} - B\boldsymbol{y}).$$

Substituting into the second row gives

$$(D - CA^{-1}B)\boldsymbol{y} = \boldsymbol{g} - CA^{-1}\boldsymbol{f}.$$

The matrix

$$D - CA^{-1}B$$

is called the Schur complement.

# 24 Complexity and practical implementation

## 24.1 Dense matrix operations

For dense matrices:

| Operation | Approximate cost | Comment |
|---|---|---|
| Matrix-vector product | $O(n^2)$ | for $n \times n$ dense matrix |
| Matrix-matrix product | $O(n^3)$ | classical algorithm |
| Gaussian elimination | $O(n^3)$ | about $2n^3/3$ flops |
| Triangular solve | $O(n^2)$ | after factorization |
| Sparse matrix-vector product | $O(\text{nnz})$ | depends on nonzeros |

### 24.2 Memory layout

Programming languages and libraries differ in memory layout:

- C and NumPy commonly use row-major layout by default.
- Fortran and many classic linear algebra libraries use column-major layout.

This affects performance because nearby memory access is faster than scattered memory access.

### 24.3 Do not reinvent everything

For serious computation, use reliable numerical libraries. Examples include BLAS, LAPACK, Eigen, SuiteSparse, SciPy, NumPy, Julia's LinearAlgebra, and similar tools.

The mathematical knowledge remains essential because libraries require the user to choose the right method and interpret results correctly.

## 25 Worked mini-project: solving a small network balance problem

Consider a directed network with three internal nodes. Let $x_1, x_2, x_3$ be unknown corrections to node potentials. Suppose the balance equations are

$$\begin{aligned} 4x_1 - x_2 - x_3 &= 2, \\ -x_1 + 3x_2 - x_3 &= 1, \\ -x_1 - x_2 + 4x_3 &= 3. \end{aligned}$$

The matrix form is

$$\begin{bmatrix} 4 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}.$$

This matrix is symmetric. Its diagonal entries dominate the row sums of the off-diagonal magnitudes, which is a favorable property for solving.

Elimination begins with

$$\left[\begin{array}{ccc|c} 4 & -1 & -1 & 2 \\ -1 & 3 & -1 & 1 \\ -1 & -1 & 4 & 3 \end{array}\right].$$

Using the first pivot gives multipliers $-1/4$ in rows 2 and 3. The reduced system can then be solved by standard elimination or by a specialized symmetric method.

The important point is not the arithmetic itself, but the pattern: network balance problems often lead to sparse, structured, symmetric matrices.

## 26 Summary of key ideas

- A matrix is both a rectangular data structure and a representation of a linear transformation.
- The system $A\boldsymbol{x} = \boldsymbol{b}$ can be interpreted by rows, columns, or transformations.
- Gaussian elimination solves systems by preserving the solution set through elementary row operations.

- Pivots reveal rank, free variables, consistency, and uniqueness.
- Invertibility connects unique solvability, full rank, independent columns, and nonzero determinant.
- LU factorization is Gaussian elimination organized for repeated computation.
- Numerical linear algebra must account for floating point errors, conditioning, and stability.
- Least squares solves inconsistent overdetermined systems by projection.
- Sparse matrices are essential in large-scale computation.
- The right matrix method depends on structure: dense, sparse, symmetric, triangular, rectangular, well-conditioned, ill-conditioned, or low-rank.

# 27 Exercises

## Basic matrix operations

1. Let
$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 & 1 \\ -1 & 2 \end{bmatrix}.$$
Compute $A + B$, $AB$, $BA$, and $A^T$.
2. Give an example of two $2 \times 2$ matrices $A, B$ such that $AB \neq BA$.
3. Let
$$A = \begin{bmatrix} 2 & 0 & 1 \\ -1 & 3 & 2 \end{bmatrix}, \qquad \boldsymbol{x} = \begin{bmatrix} 1 \\ -2 \\ 4 \end{bmatrix}.$$
Compute $A\boldsymbol{x}$.

## Linear systems

4. Solve by Gaussian elimination:
$$\begin{aligned} x + 2y &= 5, \\ 3x - y &= 4. \end{aligned}$$
5. Determine whether the system is consistent:
$$\begin{aligned} x + y + z &= 1, \\ 2x + 2y + 2z &= 3. \end{aligned}$$
6. Find all solutions:
$$\begin{aligned} x + 2y - z &= 0, \\ 2x + 4y - 2z &= 0. \end{aligned}$$
7. For which values of $a$ is the matrix
$$\begin{bmatrix} 1 & a \\ 2 & 4 \end{bmatrix}$$
invertible?

## Rank and null spaces

8. Find the rank of
$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}.$$

9. Find a basis for the null space of

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 1 & 3 \end{bmatrix}.$$

10. Suppose $A \in \mathbb{R}^{5\times 8}$ has rank 5. What is $\dim \operatorname{Null}(A)$?

## Computation and interpretation

11. Explain why solving $A\boldsymbol{x} = \boldsymbol{b}$ by explicitly computing $A^{-1}$ is usually not the preferred numerical method.
12. Suppose a sparse matrix has size $10^6 \times 10^6$ and about 7 nonzero entries per row. Estimate the number of stored nonzero entries.
13. Describe the least squares problem for fitting a quadratic model

$$y \approx c_0 + c_1 t + c_2 t^2$$

to data points $(t_i, y_i)$.
14. In a graph with adjacency matrix $A$, what does the entry $(A^3)_{ij}$ represent?

## Compact formula sheet

| Concept | Formula / condition |
|---|---|
| Matrix-vector product | $(A\boldsymbol{x})_i = \sum_j a_{ij}x_j$ |
| Matrix multiplication | $(AB)_{ij} = \sum_k a_{ik}b_{kj}$ |
| Transpose of product | $(AB)^T = B^T A^T$ |
| Column view | $A\boldsymbol{x} = x_1\boldsymbol{a}_1 + \cdots + x_n\boldsymbol{a}_n$ |
| Consistency | $\operatorname{rank}(A) = \operatorname{rank}([A \mid \boldsymbol{b}])$ |
| Unique solution | consistent and $\operatorname{rank}(A) = n$ |
| Rank-nullity | $\operatorname{rank}(A) + \dim \operatorname{Null}(A) = n$ |
| Inverse solution | $\boldsymbol{x} = A^{-1}\boldsymbol{b}$, if $A^{-1}$ exists |
| Determinant test | $A$ invertible iff $\det(A) \neq 0$ |
| LU solve | $A = LU$, solve $L\boldsymbol{y} = \boldsymbol{b}$, then $U\boldsymbol{x} = \boldsymbol{y}$ |
| Least squares | minimize $\|A\boldsymbol{x} - \boldsymbol{b}\|_2^2$ |
| Normal equations | $A^T A\hat{\boldsymbol{x}} = A^T\boldsymbol{b}$ |
| Condition number | $\operatorname{cond}(A) = \|A\| \, \|A^{-1}\|$ |

The central question is always: what structure does the matrix have, and what does that structure imply for solvability, uniqueness, stability, memory, and algorithmic cost?